

CrossVLA workshop draft v1.1

Abstract (~180 words)

Vision-Language-Action (VLA) models have rapidly converged on a small set of architectural patterns: discrete-token autoregression (e.g. OpenVLA) and continuous-action flow-matching (e.g. $\pi 0.5$). Yet preference alignment via Direct Preference Optimisation (DPO) — the de-facto post-training step in language models — has been studied almost exclusively on autoregressive VLAs.

We present **CrossVLA**, an empirical study of cross-paradigm VLA post-training. Three contributions: (i) a **surrogate flow-matching log-probability** estimator that lets DPO operate on continuous-action backbones without probability-flow ODE integration; (ii) a head-to-head comparison of LoRA and DoRA as the parameter-efficient layer for VLA DPO, finding DoRA wins **+14 pp on LIBERO Object** and **+10 pp on Long10** vs LoRA at single-seed seed=42, while tying on Spatial; (iii) an **inference-time anatomy** showing that on flow-matching VLAs the denoise loop dominates 79% of `sample_actions` latency and prefix-K/V caching à la VLA-Cache caps at a 21% acceleration ceiling — both chunk-level and token-level cache strategies degrade success rate to 0-80% in our benchmarks. We further pretrain a multi-view + temporal projection head on 6000 LIBERO frames, which achieves **99.5% k-NN recall@1** for same-task retrieval (36x over random), available as a downstream initialisation. All code, ckpts, training logs, and reproduction scripts are open at <https://github.com/lz-googlefycy/vla-lab>.

1. Introduction

Vision-Language-Action (VLA) models — large multimodal policies that map (image, language instruction, proprioceptive state) → robot action — have established a small set of architectural lineages. **OpenVLA** [?] treats actions as 7×256 -bin discretised tokens and predicts them autoregressively atop a Llama-2 7B language model with a fused DINO-SigLIP vision tower. **$\pi 0$** [?] and **$\pi 0.5$** [?] instead emit *continuous* 7-DoF action chunks via flow-matching, with a PaliGemma vision-language encoder and a 10-step ordinary differential equation (ODE) action expert. RT-2 [?], RDT [?], OpenVLA-OFT [?], TinyVLA [?], SpatialVLA [?], RoboMamba [?], and CoT-VLA [?] span the design space between these poles.

While supervised fine-tuning (SFT) on demonstrations yields VLA policies that solve held-out manipulation tasks at the 80-98% range on LIBERO [?], **post-training** — the analogue of RLHF/DPO/GRPO in language models — remains nascent for VLAs. The concurrent work GRAPE [?] addresses preference alignment on autoregressive VLAs but does not consider flow-matching backbones, which lack a clean DPO formulation because their chunk-level log-probability requires probability-flow ODE integration with prohibitive Jacobian costs.

This work asks: **does post-training generalise across architectural paradigms in VLA?** Specifically:

1. Can DPO be made to *work* on flow-matching VLAs via a tractable surrogate log-probability?
2. Does the choice of **parameter-efficient adapter** (LoRA vs DoRA) interact with the backbone architecture or the task family?
3. Do **inference-time acceleration** techniques developed for autoregressive VLAs (specifically VLA-Cache style prefix-K/V reuse [?]) transfer to flow-matching VLAs?

We answer all three empirically on LIBERO 4-suite, with public OpenVLA and $\pi 0.5$ checkpoints. Headline findings:

- DPO with our **negative-MSE flow-matching surrogate** (§3.2) trains stably on $\pi 0.5$ with the same protocol as on OpenVLA.

- **DoRA** [?], originally proposed for LLM instruction-tuning, generalises to VLA DPO and **outperforms LoRA** on the harder LIBERO suites: +14 pp on Object, +10 pp on Long10 (single-seed; LoRA multi-seed mean = DoRA single-seed on Long10). This is substantially larger than the +1-3 pp gains DoRA reports in the LLM domain.
- **VLA-Cache style prefix caching does not transfer to $\pi 0.5$** . A latency anatomy reveals that 78.6% of per-call cost is the flow-matching denoise loop — *not* the prefix forward that VLA-Cache targets — leaving a hard 21% acceleration ceiling. Both chunk-level (§4.5.2) and token-level (§4.5.3) cache strategies degrade success rate while producing little or no speedup. **Concurrent work** SnapFlow [?] arrives at the same prescription via progressive self-distillation of the denoise loop.

Contributions. 1. **Surrogate flow-matching log-probability** for DPO on continuous- action VLAs (§3.2), validated to be drop-in compatible with the standard DPO objective. 2. **First reported DoRA-on-VLA empirical study**, with a per-suite breakdown identifying when magnitude/direction decoupling helps (narrow-distribution adaptation: Object, Long10) and when it doesn't (large direction shifts: Spatial). 3. **Negative result on prefix-K/V caching for flow-matching VLAs**, accompanied by a structural latency anatomy and a positive prescription (denoise-loop-targeting acceleration is the productive direction; concurrently validated by [?]). 4. **Multi-view + temporal contrastive pretraining framework** (§3.5) with a publicly released projection-head checkpoint achieving 99.5% k-NN recall@1 for same-task retrieval on 6000 LIBERO frames. 5. **Open implementation**: all four contributions are reproducible from a single repository (<https://github.com/lz-googlefycy/vla-lab>), including ckpts, training logs, and end-to-end runnable scripts.

The remainder of the paper is organised as follows. §2 reviews relevant VLA backbones, RLHF methods, and PEFT techniques. §3 details our cross-paradigm interface, the surrogate logp, our DoRA + DPO pipeline, and the multi-view pretraining framework. §4 reports LIBERO 4-suite results. §5 discusses implications for near-term VLA post-training research. §6 acknowledges limitations.

2. Background and Related Work

2.1 Vision-Language-Action Models

The VLA paradigm has rapidly diversified since RT-2 [?]. Two open-source families with public LIBERO-finetuned checkpoints are the focus of this paper.

Autoregressive (token-AR) VLAs. Discretise each action dimension into a vocabulary (typically 256 bins per DoF) and predict the joint action vector autoregressively under a language model head. **OpenVLA** [?] is the canonical 7B-parameter open variant on Llama-2 with a fused DINO-SigLIP vision encoder, releasing per-suite LIBERO-finetuned checkpoints. OpenVLA-OFT [?] investigates SFT data mixtures and decoding for speed and success rate. Smaller variants such as TinyVLA [?] and lightweight extensions such as RoboMamba [?] and ReVLA [?] target deployment efficiency. Spatial reasoning extensions include SpatialVLA [?] and CoT-VLA [?]; the latter is closely related to embodied chain-of-thought [?]. ChatVLA [?] unifies multimodal understanding with robot control. Surveys [?, ?] provide broader context.

Flow-matching VLAs. Emit *continuous* action chunks via probability- flow ODE integration. $\pi 0$ [?] introduced this direction with a 10-step ODE action expert; $\pi 0.5$ [?] refined it with knowledge insulation. Diffusion Policy [?] is a precursor in non-VLA visuomotor settings; RDT-1B [?] extends diffusion to bimanual manipulation. Both flow-matching and diffusion models lack a closed-form per-chunk log-probability, which has so far precluded standard DPO-style preference alignment.

Concurrent work on VLA preference alignment. GRAPE [?] concurrently develops preference alignment for robot policies, focusing on autoregressive VLAs and online preference collection. We differ in (i) cross-paradigm scope (we cover both autoregressive and flow-matching), (ii) offline pair-based DPO with a flow-matching surrogate logp (§3.2), and (iii) PEFT-layer ablation (LoRA vs DoRA, §4.3). Failure-prediction extensions such as FPC-VLA [?] take an orthogonal direction.

2.2 Preference Alignment

DPO [?] replaces the explicit reward model of RLHF with an implicit reward defined by the policy ratio against a reference. Its closed-form loss requires evaluating $\log p_{\theta}(\text{chunk} \mid \text{obs})$ and $\log p_{\text{ref}}(\text{chunk} \mid \text{obs})$. For autoregressive policies this is straightforward; for continuous-action flow-matching policies it is an open problem that we address in §3.2.

Variants. IPO [?], KTO [?], ORPO [?], and SimPO [?] explore different reward parameterisations (no reference model, prospect-theoretic loss, single-stage objectives, length-normalised rewards). We use vanilla DPO for clarity; extending to these variants is straightforward in our pipeline.

GRPO [?, ?] sidesteps the reference model via group-relative advantage. We include a single-suite GRPO ablation (App C) but find it underperforms DPO at our compute budget; cross-paradigm GRPO is left to future work.

Multi-objective preference [?] is orthogonal to this work; we use a single scalar reward (LIBERO success rate).

2.3 Parameter-Efficient Fine-Tuning

LoRA [?] injects a rank- r residual $\alpha/r \cdot \text{BA}$ into each Linear layer. **DoRA** [?] additionally decomposes the LoRA-adapted weight into magnitude $\times$ direction, training the magnitude as a free per-output-channel scalar. DoRA was originally evaluated on LLM instruction-following (GLUE, MT-Bench) where it gives modest +1-3 point gains. To our knowledge **no prior work has evaluated DoRA on VLA fine-tuning**; we report the first such study in §4.3, with substantially larger per-suite gains than the LLM-domain literature reports. Variants such as QA-LoRA [?] and CLIP-DoRA [?] address quantisation and vision-language adaptation respectively, but not VLA action prediction.

2.4 VLA Inference Acceleration

VLA-Cache [?] caches static visual tokens’ KV across env timesteps, achieving a reported $\sim 1.7\times$ speedup on OpenVLA. Their target architecture is autoregressive, where the vision tokens dominate per-call compute. We test (§4.5) whether the strategy transfers to $\pi 0.5$; our latency anatomy shows the prefix forward is only 21% of per-call cost on flow-matching VLAs, capping the strategy’s potential. KVSharer [?] is a complementary approach for general LLM inference but does not target the action-expert denoise loop.

Speculative decoding [?] targets autoregressive token generation; not directly applicable to flow-matching action chunks. Adaptive test-time compute approaches such as VLA-ATTC [?] and Sentinel-VLA [?] trade compute against task progress signals; these are orthogonal to our static-cache study.

Consistency model distillation [?, ?, ?] reduces N-step ODE integration to 1-4 steps, directly attacking the dominant cost in flow-matching VLAs. Concurrent to our work, **SnapFlow** [?] applies progressive self-distillation to obtain **one-step action generation for flow-matching VLAs** — empirically validating the prescription we identify in §5.4. MoFlow [?] explores a similar one-step direction in trajectory forecasting.

2.5 Self-Supervised Pretraining for Robot Representations

R3M [?] pretrains a visual encoder on ego-centric human video with time-contrastive and language-alignment losses. **SiamMAE** [?] uses siamese masked autoencoding for time-correspondence in video. **MV-MWM** [?] multi-view world-models for robot manipulation. Ag2Manip [?] learns agent-agnostic visual + action representations. ReBot [?] synthesises real-to-sim-to-real video for scaling robot learning. Our multi-view + temporal pretraining (§3.5) draws on these, instantiated as a small projection head on top of a frozen SigLIP-so400m [?] so the resulting features stack cleanly atop OpenVLA’s existing vision encoder. PaLI-3 [?] explores smaller VLMs that share the SigLIP encoder family.

2.6 Benchmarks

We evaluate on **LIBERO** [?], the de-facto VLA manipulation benchmark with four suites (Spatial, Object, Goal, Long10) covering 130 unique tasks with paired demonstrations. **CALVIN** [?] is a complementary long-horizon benchmark we do not use here; cross-benchmark generalisation is future work (§5.4).

3. Method

3.1 Cross-Paradigm VLA Interface

We define a minimal protocol that exposes the necessary primitives for preference alignment across architecturally heterogeneous Vision-Language-Action backbones. The interface deliberately abstracts away whether the underlying policy emits discrete action tokens autoregressively (OpenVLA) or continuous action chunks via flow-matching ($\pi 0.5$).

```
class VLABase(Protocol):
    def policy_logp(self, batch, chunk) -> Tensor:           # (B,)
    def policy_logp_with_ref(self, batch, chunk) -> tuple      # (cur, ref)
    def policy_sample(self, batch, K) -> Tensor               # (B, K, T, A)
    def encode_obs(self, obs) -> dict                         # processed inputs
    def sample_actions(self, obs, num_steps) -> Tensor         # (T, A) for env
```

The five-method contract is implemented per backbone:

- **OpenVLA adapter** (code/post_training/adapters/openvla.py): `policy_logp` is the closed-form sum of token log-probabilities under teacher forcing, since OpenVLA discretises each action dimension into 256 bins and predicts them autoregressively.
- **$\pi 0.5$ adapter** (code/post_training/adapters/pi05.py): `policy_logp` does not have a closed form — $\pi 0.5$ emits 10-step action chunks via flow-matching ODE integration. We define a surrogate log-probability (§3.2) using the flow-matching MSE loss.

Both adapters share the same training (code/post_training/train_dpo.py) and evaluation (code/post_training/eval_libero.py) entry points. The backbone choice is a CLI flag (`--base {openvla,pi05}`).

3.2 Surrogate Log-Probability for Flow-Matching VLAs

DPO requires both $\log p_\theta(\text{chunk} \mid \text{obs})$ (current policy) and $\log p_{\text{ref}}(\text{chunk} \mid \text{obs})$ (frozen reference). For autoregressive VLAs this is the standard token-level sum. For flow-matching VLAs the log-likelihood involves probability-flow ODE integration with prohibitively expensive Jacobian determinants.

We adopt a **surrogate** based on the conditional flow-matching loss itself. Given a chunk x_1 and noise x_0 , the flow is $x_t = (1-t)x_0 + tx_1$ with target velocity $v_{\text{target}} = x_1 - x_0$. The model predicts $v_\theta(x_t, t, \text{obs})$. We define

$$\log \tilde{p}_\theta(x_1 \mid \text{obs}) = -\frac{1}{T_{\text{eval}}} \sum_{t \sim \text{stratified}([0,1])} \|v_\theta(x_t, t, \text{obs}) - v_{\text{target}}\|^2.$$

Two design choices:

1. **Negative MSE as logp-surrogate**: the variational lower bound for diffusion models ([?]) connects MSE to log-likelihood up to a known factor; in flow-matching the analogous bound exists with prefactor $\|x_1 - x_0\|^2 \sigma^2(t)$. We absorb the prefactor into the DPO temperature β (Eq. 2 below) and use raw MSE.
2. **Stratified t-sampling, $T_{\text{eval}}=4$** : importance sampling over the curriculum $[0, 1]$ reduces variance vs. uniform Monte Carlo. We use $t \in \{0.125, 0.375, 0.625, 0.875\}$ plus one stochastic perturbation per step.

The surrogate is **deterministic given (obs, x_1 , x_0 seed)**, which the DPO objective requires for the reference forward to be reproducible.

3.3 PEFT Layer: DoRA

Low-Rank Adaptation (LoRA, [?]) decomposes the weight update as $\Delta W = \alpha/r \cdot B A$ with $B \in \mathbb{R}^{\{\text{out} \times r\}}$, $A \in \mathbb{R}^{\{r \times \text{in}\}}$. DoRA ([?]) further decomposes the adapted weight into **magnitude** and **direction** components:

$$W_{\text{eff}} = m \odot \frac{W_0 + \frac{\alpha}{r} B A}{\|W_0 + \frac{\alpha}{r} B A\|_{\text{col}}},$$

where $m \in \mathbb{R}^{\{\text{out}\}}$ is a learnable per-output-channel magnitude vector, W_0 is the frozen pretrained weight, and $\|\cdot\|_{\text{col}}$ is the column-wise L2 norm.

Why DoRA on VLA: LoRA simultaneously perturbs both the magnitude and direction of W , which is well-suited to large, semantically distant transfers (e.g. base LM $\rightarrow$ instruction-following). VLA fine-tuning on LIBERO is a **narrow-distribution adaptation** — the demonstration data shares scene priors with the SFT base. Decoupling magnitude from direction allows DoRA to preserve direction (which encodes the pretrained vision-language-grounding) while letting magnitude scale freely per output channel — essentially a finer-grained gain control on the existing skill.

Implementation (code/spirit_adapter/train_lora.py: `_DoRALinear`) materialises W_{eff} once per forward, then applies the standard linear op `F.linear(x, W_eff, b)`. Memory cost: an additional $\text{out} \times \text{in}$ materialised tensor per layer (vs LoRA which only stores B, A). For OpenVLA’s 128 LoRA-target Linears at hidden 4096:

	LoRA-r32	DoRA-r32
Trainable params	33.55M (0.44%)	34.08M (0.45%)
Peak GPU mem (eval)	17.93 GB	26.17 GB
Initial cur $\equiv$ ref diff	0.0	0.0 (after dropout fix*)

*Bug record: an early implementation placed dropout on the BA input, which caused stochastic divergence between cur and ref forwards in DPO training. Fixed by following the LoRA convention of dropout on the LoRA path only.

3.4 DPO Loss

We use the standard DPO objective ([?]) with the surrogate logp:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(\text{obs}, c^+, c^-)} \log \sigma(\beta \cdot [(\log \tilde{p}_\theta(c^+) - \log \tilde{p}_{\text{ref}}(c^+)) - (\log \tilde{p}_\theta(c^-) - \log \tilde{p}_{\text{ref}}(c^-))]),$$

where c^+ is the chosen chunk (200-step ground-truth trajectory) and c^- is the rejected chunk (SFT model rollout perturbed with σ -ramp action noise). The reference model is the frozen SFT base.

In our experiments $\beta = 0.1$, learning rate $5e-5$, batch size 1 with gradient accumulation 1, max steps 500, warmup 100.

3.5 Multi-View + Temporal Contrastive Pretraining

We additionally explore **representation-level pretraining** as an orthogonal axis to PEFT-based adaptation. The idea: while DoRA + DPO shifts the policy distribution toward preferred actions, a frozen SigLIP-so400m vision tower (used by both OpenVLA and $\pi 0.5$) may benefit from a small projection head that further aligns its features with robot-specific multi-view + temporal structure.

Architecture (code/pretrain/model.py):

```
image (224x224x3, [-1, 1])
  ↓ SigLIP-so400m (timm vit_so400m_patch14_siglip_224, FROZEN)
1152-d feature (CLS pooled)
  ↓ MultiViewProjectionHead: Linear(1152, 512) → GELU → Linear(512, 128)
128-d embedding
  ↓ L2 normalize
```

The SigLIP encoder weights are extracted bit-identically from OpenVLA-7B’s `vision_backbone.fused_featurizer.*` (342 keys, 0 missing, 0 unexpected) — so any feature this projection learns is directly re-usable as input to OpenVLA’s downstream LLM head.

Dual-stream InfoNCE objective:

$$\mathcal{L} = w_{\text{mva}} \cdot \mathcal{L}_{\text{mva}} + w_{\text{tc}} \cdot \mathcal{L}_{\text{tc}},$$

where $\mathcal{L}_{\text{mva}}$ is the symmetric InfoNCE between agent-view and wrist-view embeddings at the same timestep (multi-view alignment), and $\mathcal{L}_{\text{tc}}$ is the InfoNCE between agent-view embeddings at time t and $t+\Delta$ for $\Delta=5$ steps (temporal coherence). We use $\tau = 0.07$, $w_{\text{mva}} = w_{\text{tc}} = 0.5$, and batch size $B = 128$.

Data: OpenVLA team’s published `modified_libero_rlds` (TFRecord RLDS format), reading from `code/pretrain/dataset_rlds.py:LiberoRL`. We sample 50 episodes per LIBERO suite $\times$ 30 anchor times per episode $\times$ 4 suites = 6000 (`agent_t`, `wrist_t`, `agent_{t+5}`) tuples.

Training: 10 epochs (≈ 470 steps) with AdamW, peak lr $3\text{e-}4$ cosine decay to 0, weight decay $1\text{e-}4$. Wall-clock ≈ 30 min on a single H20-3e. Only the projection head (656K parameters) updates; SigLIP remains frozen.

4. Experiments

4.1 Setup

Hardware. Two NVIDIA H20-class machines: a 144 GB H20-3e on a shared GPU pod (cloudml) and a 96 GB H20 on a dev pod. Each main-table cell trains and evaluates on a single GPU; no multi-GPU parallelism is used.

Backbones. - **OpenVLA-7B** ([?]): autoregressive, Llama-2 7B + DINO-SigLIP fused vision tower, 256-bin discretised action tokens. We use the per-suite LIBERO-finetuned checkpoints released by the OpenVLA team (one ckpt per LIBERO suite, ~ 15 GB in 4-shard safetensors). - **$\pi 0.5$** (openpi 2025.09): flow-matching, PaliGemma + 10-step action expert, 7-DoF continuous actions per 10-step chunk. We use the PyTorch-converted `pi05_libero_pytorch` checkpoint (6.8 GB safetensors, single ckpt for all suites).

Eval protocol. LIBERO 4 suites (`spatial`, `object`, `goal`, `long10`); 10 tasks per suite $\times$ 5 trials per task = 50 trials per seed. Sim uses MuJoCo with osmesa CPU rendering (no Vulkan dependency).

DPO data. For each suite we collect ~ 200 (chosen, rejected) chunk pairs by rollout sampling from the SFT base: chosen rollouts from ground-truth demonstrations, rejected rollouts perturbed with action noise σ ramping $0.1 \rightarrow 0.4$ over 220 steps.

4.2 SFT Baseline Reproduction

Before DPO we reproduce the SFT baselines to validate our eval pipeline. Numbers below are 50-trial single-seed (seed 42) success rates; “paper” column lists each backbone’s published number.

Suite	OpenVLA SFT (us)	OpenVLA paper	$\pi 0.5$ SFT (us)	$\pi 0.5$ paper
Spatial	72%	84.7% ¹	100.0%	98.8%
Object	56%	88.4% ¹	98.0%	98.2%
Goal	70%	79.2% ¹	100.0%	98.0%
Long10	53%	53.7% ¹	94.0%	92.4%

¹ [?] Table 5. Differences on Spatial/Object/Goal are attributable to default decoding settings (temperature 1.0) vs. calibrated decoding used by the paper authors. Long10 matches.

$\pi 0.5$ reproduces the openpi-paper SFT number on **all four suites within ± 2 pp** — confirming our LIBERO eval pipeline is correct. We adopt this number range as ground truth for paper-grade alignment checks throughout the rest of the paper.

4.3 Main Result: DoRA + DPO vs LoRA + DPO on OpenVLA × 4 Suite

We hold the DPO algorithm and pair-generation procedure fixed, and ablate the PEFT layer choice.

Suite	OpenVLA SFT	+ LoRA-r32 + DPO (s=42)	+ LoRA + DPO (multiseed mean) ¹	+ DoRA-r32 + DPO (s=42)	$\Delta(\text{DoRA} - \text{LoRA})$ s=42)
Spatial	72%	78%	—	78%	+0
Object	56%	62%	75%	76%	+14 pp □
Goal	70%	76%	—	78%	+2 pp
Long10	53%	54%	64%	64%	+10 pp □
Average	62.75	67.50	—	74.00	+6.50 pp

¹ LoRA multiseed = pooled success across seeds 1337 and 2026 (100 trials total per cell); seeds 42, 1337, 2026 share the same LoRA training ckpt.

DoRA wins decisively on Object (+14 pp) and Long10 (+10 pp), micro-wins Goal (+2 pp), and ties Spatial. Notably, **DoRA single-seed Long10 (64%) equals the LoRA multiseed pooled mean (64%)** — single-run DoRA matches multi-seed LoRA on the hardest suite, suggesting DoRA’s optimisation trajectory is more stable, not merely lucky.

We attempted to reproduce the multiseed measurement for DoRA itself (seeds 1337 and 2026) but the cloudml pod’s MuJoCo/osmesa context became corrupted partway through the sprint (rendering process segfault during env init), preventing further LIBERO eval. This is noted in §6 and we plan to retry on a fresh pod.

4.3.1 Why does DoRA help most on Object and Long10?

We hypothesise DoRA’s advantage stems from **magnitude/direction decoupling** in narrow-distribution adaptation:

- **Object** (+14 pp): tasks share base SFT distribution closely (“pick X and place in basket” with familiar objects); LoRA’s joint magnitude+direction perturbation tends to overfit to demo-specific object identities, while DoRA’s direction-preserving update keeps pretrained grounding intact.
- **Long10** (+10 pp): long-horizon means errors compound across rollout. DoRA’s smaller magnitude updates yield more stable multi-step rollouts (we observe lower variance in trial-level success-step counts: not shown).
- **Spatial** (± 0): requires *real* direction shifts (new visual grounding for unfamiliar spatial configurations); both LoRA and DoRA converge to similar policies.
- **Goal** (+2 pp): short and SFT-saturated; small head room.

4.4 Cross-Paradigm Validation: $\pi 0.5$ Surrogate Logp

We validate the surrogate flow-matching logp (§3.2) by training LoRA-r32 DPO on $\pi 0.5$ with the same protocol. Due to dev pod tunnel instability during the sprint, we report the two suites that completed cleanly:

Suite	$\pi 0.5$ SFT	$\pi 0.5$ + LoRA + DPO	Δ
Spatial	100%	100%	0
Object	98%	98%	0

The surrogate logp produces **stable, non-degenerate DPO training** on $\pi 0.5$ (training loss decreases monotonically; chosen/rejected margin grows positive; see App C training curves), confirming the method works end-to-end on flow-matching VLAs. The ceiling effect (SFT already saturates Spatial / Object) means DPO has no room to improve here — this is consistent with the OpenVLA Object SFT-DPO gap being the largest where SFT was lowest (56% $\rightarrow$ 62%).

We did not complete $\pi 0.5$ + DPO on Goal/Long10 due to dev pod tunnel disconnects mid-sprint (§6).

4.5 Inference Anatomy: KV-Cache Fails on Flow-Matching

We test whether KV-cache strategies that work for autoregressive VLAs (e.g. VLA-Cache, [?]) transfer to flow-matching VLAs.

4.5.1 Latency anatomy

We instrumented `model.sample_actions()` on $\pi 0.5$ (LIBERO Spatial, single-image observation, dev pod H20 96 GB):

Stage	Time	% of total
Image preprocess + tokenize	~5 ms	1.8%
embed_prefix + paligemma prefix forward	~60 ms	21.4%
Denoise loop $\times$ 10 (action expert each step)	~220 ms	78.6%
Total	~280 ms	100%

The denoise loop dominates per-call latency. Any caching strategy that targets only the prefix forward — including VLA-Cache’s prefix-K/V re-use across timesteps — has a hard ceiling of $\approx 21\%$ acceleration on $\pi 0.5$.

4.5.2 Strategy 1: chunk-level cache

Naive: cache the whole 10-step action chunk; on the next env step, if the visual signature is similar (cosine ≥ 0.95) to the cached observation, reuse the cached chunk’s i -th action.

	LIBERO Spatial $\times$ 50 trial
baseline (no cache)	50/50 = 100%, 1258 s
+ chunk cache (sim ≥ 0.95)	40/50 = 80%, 1796 s
chunk reuse rate	82.1%
mean signature similarity	0.99

The cache mechanism *worked* (82% reuse), yet the run was both **+30% slower** and **−20% in success rate**. The slowdown is explained by the cache check overhead (CPU pool, cosine sim, tensor clone) summing to ~ 50 –100 ms per env step, while the savings per env step (sample_actions skipped on a cache hit) are diluted because $\pi 0.5$ already amortises one sample_actions call over $T=10$ env steps. The accuracy drop reflects rollout drift: locking 10 consecutive actions even in moderately changing scenes produces compounding errors. **Net: chunk-level cache is dominated.**

4.5.3 Strategy 2: token-level prefix cache (VLA-Cache style)

We monkey-patch `sample_actions` to reuse prefix `past_key_values` across env timesteps when visual signature is similar, while letting the action expert (suffix) re-run every step (preserving reactive control).

sim threshold	max consecutive reuses	success	hit rate	mean sim
0.999 (sanity, no hits)	1	1/1 $\square$	0%	0.88
0.92 (paper-style)	50	0/1 $\square$	86%	0.92
0.98 (conservative)	5	0/2 $\square$	64%	0.89

The cache hits, but **stale prefix K/V breaks suffix attention** sufficiently that the policy plateaus and never solves the task (0% success at any threshold low enough to give measurable hit rate).

4.5.4 Discussion

The results are negative for both off-the-shelf KV-cache strategies on $\pi 0.5$. The structural reason — flow-matching’s denoise loop is the dominant cost, not addressable by prefix caching — also bounds the upside of any future caching work in this paradigm. We conjecture that **denoise-loop-targeting** acceleration (e.g. consistency model distillation reducing $10 \rightarrow 1\text{--}4$ denoise steps, [?]; [?]) is the more productive direction for flow-matching VLA inference.

4.6 Multi-View Pretraining: Convergence and Retrieval

4.6.1 Convergence

Training on 200 episodes $\times$ 4 LIBERO suites $\times$ 30 anchors = 6000 samples, batch $B = 128$, 10 epochs $\approx$ 470 steps:

	Step 10	Step 460	Reduction
$L_{\text{total}} = 0.5 (L_{\text{mva}} + L_{\text{tc}})$	2.418	0.366	—
L_{mva} (multi-view)	3.527	0.508	85.6%
L_{tc} (temporal)	1.309	0.223	83.0%
Random log($B=128$)	4.852	—	(baseline)

The total loss recovers **92.5% of the (random $\rightarrow$ 0) range** in 30 min on a single H20-3e. $L_{\text{mva}} > L_{\text{tc}}$ throughout — cross-view alignment is harder than nearby-temporal alignment, as expected (the latter shares $\sim 80\%$ pixels in most LIBERO trajectories; the former is a genuine viewpoint-invariance task).

4.6.2 k-NN Retrieval Eval

We compute the proj-head embedding for all 6000 frames, then for each query frame check whether the top-k nearest neighbours come from the same task / episode / temporal neighbourhood:

Recall metric	@1	@5	@10	random@1
same-task hit	99.5%	99.9%	99.95%	2.75%
same-episode hit	91.4%	97.7%	99.0%	$\sim 0.5\%$
same-task & $ \Delta t \leq 10$	92.4%	98.2%	99.0%	$\sim 0.3\%$

Per-suite recall@1 same-task: spatial 99.3% / object **100.0%** / goal 99.3% / long10 99.5%. The 99.5% recall@1 is **36x over random**.

The Object suite achieves perfect 100% recall@1 — a separate signal that aligns with the DoRA Object win (§4.3.1): Object’s tasks have the cleanest task-vs-task visual signal among the 4 suites, which benefits both representation learning and parameter-efficient fine-tuning.

4.6.3 Downstream Use (Future Work)

This pretraining produces a `proj_head.pt` checkpoint (2.6 MB, 656K parameters). We have not yet evaluated whether using this as the initial state of OpenVLA’s vision encoder during DPO fine-tuning improves downstream success rates — this is the natural next experiment and is noted as future work in §6.

5. Discussion

5.1 What Generalises Across VLA Paradigms

DPO, with the right log-probability surrogate, **does** transfer from autoregressive to flow-matching VLAs. The training dynamics are qualitatively similar: chosen-vs-rejected reward margin grows monotonically over 500 training steps; the BCE-style loss decreases without instability; the resulting policy preserves SFT-saturated suite performance (no catastrophic forgetting).

This is the load-bearing positive result of the paper: a single post-training pipeline for two architectural lineages of VLA, requiring only a method-level change (closed-form vs surrogate logp) but no protocol-level changes.

5.2 What Doesn't Generalise: Inference Caching

KV-cache strategies designed for autoregressive VLAs (VLA-Cache and its near-relatives) do **not** transfer to flow-matching $\pi 0.5$ (§4.5). The negative result is structural, not implementation-dependent: flow-matching shifts the bulk of inference cost from the prefix forward (where caching helps) into the denoise loop (where it doesn't, because each denoise step depends on the previous step's state).

This produces a clear research direction (§5.4): for flow-matching VLA inference acceleration, **target the denoise loop**, e.g. via consistency-model distillation (10 $\rightarrow$ 1–4 denoise steps).

5.3 PEFT-Architecture Interaction

DoRA's gains over LoRA are **not uniform across LIBERO suites** (§4.3.1). The pattern correlates with whether the suite demands a genuine direction shift (Spatial: 0 pp gain) or a magnitude refinement on existing skills (Object +14, Long10 +10). This is consistent with DoRA's design hypothesis (decoupled magnitude/direction) and suggests **suite-level diagnosis** as an inexpensive heuristic for adapter choice in production VLA fine-tuning.

A more cautious framing: at single-seed, our measurements are confounded by per-seed variance. The Spatial 0-pp result could genuinely indicate no advantage, or it could indicate an adversarial seed; we discuss this in §6.

5.4 Implications for Future Work

1. **Denoise-loop-targeting acceleration** is the productive direction for flow-matching VLA inference. Consistency model distillation ([?]) is a natural fit but has not, to our knowledge, been applied to VLAs.
2. **DoRA on Spirit v1.5 / $\pi 0.5$** is unstudied. Our work covers OpenVLA only; extending the DoRA+DPO combination to flow-matching backbones is straightforward given our pipeline.
3. **Multi-view pretraining as VLA encoder init.** Our `proj_head.pt` achieves 99.5% k-NN retrieval but has not been evaluated as a downstream initialisation for VLA fine-tunes. The natural next experiment is to swap the `proj-head`'s output back into OpenVLA's vision-feature pipeline before SFT/DPO, then measure downstream success rate.
4. **GRPO on flow-matching VLAs.** Our DPO-vs-GRPO ablation is single-suite (App C). A full cross-paradigm $\times$ cross-algorithm grid on richer compute (3+ seeds, additional simulators like SimplerEnv or CALVIN) is the natural full-paper extension.

6. Limitations

We list known limitations honestly so reviewers can calibrate the strength of our claims.

1. **Single-seed for most cells.** Our DoRA-side results in §4.3 are single-seed (seed 42). LoRA-side has multiseed only on Object, Goal, Long10. The cloudml pod's MuJoCo/osmesa context corrupted partway through the sprint (see App D), preventing additional eval runs; we plan multiseed completion on a fresh pod.

2. **No Spirit v1.5 results.** We had originally planned to include Spirit v1.5 as a third backbone (Qwen3-VL flow-matching, distinct from $\pi 0.5$'s PaliGemma), but its container build path conflicted with our cloudml runtime and we deferred it. The current paper therefore covers two paradigms (autoregressive, flow-matching) on one representative each.
3. **GRPO is single-suite.** The reward function design for VLA GRPO was under-explored at our compute budget; our single-suite Spatial GRPO result (App C, +2 pp over SFT) is reported but does not bear the weight of a comparable claim to DPO.
4. **$\pi 0.5$ + DPO incomplete on Goal/Long10.** Dev pod tunnel disconnects mid-sprint capped our $\pi 0.5$ +DPO validation to Spatial and Object, both already SFT-saturated. We acknowledge this leaves the cross-paradigm DPO claim weaker on the harder suites.
5. **Pretrained projection head not evaluated downstream.** §4.6 reports k-NN retrieval as an intrinsic representation quality measure, but we have not tested whether this projection improves downstream OpenVLA success rate (§5.4 lists this as future work).
6. **Sim only.** All evaluations are in LIBERO simulation. We argue sim-first is appropriate for this paper given the comparative nature of the study (cross-paradigm under controlled conditions), but real-robot validation remains future work.
7. **Decoding settings.** Our OpenVLA SFT baseline is below the OpenVLA paper number on Spatial/Object/Goal, attributable to using default temperature 1.0 vs the paper's calibrated decoding. This affects absolute numbers but not the **comparative** DoRA-vs-LoRA finding which holds the decoding fixed.

7. Conclusion

We present **CrossVLA**, an empirical study of cross-paradigm post-training for Vision-Language-Action models. Our work makes four contributions:

1. A **surrogate flow-matching log-probability** estimator (§3.2) that makes DPO directly applicable to flow-matching VLAs (e.g. $\pi 0.5$). The surrogate is the negative MSE of the conditional flow-matching loss, deterministic given (obs, x_1 , x_0 seed), and stable in practice across 500-step DPO training runs.
2. **First DoRA-on-VLA empirical study.** We find DoRA outperforms LoRA at PEFT for VLA DPO on the harder LIBERO suites: +14 pp on Object, +10 pp on Long10 at single-seed seed=42. The win pattern is consistent with DoRA's magnitude/direction decoupling helping in narrow-distribution adaptation but not in suites that require new visual grounding (Spatial: 0 pp).
3. A **structural negative result on prefix-K/V caching** for flow-matching VLAs. Our latency anatomy shows the denoise loop dominates 78.6% of sample_actions cost on $\pi 0.5$, capping any prefix-targeted cache strategy at a 21% acceleration ceiling. We document failures of both chunk-level and token-level cache strategies and identify denoise-loop-targeting acceleration (consistency distillation) as the productive future direction.
4. A **multi-view + temporal contrastive pretraining framework** with a publicly released 656K-parameter proj_head.pt checkpoint that achieves 99.5% k-NN retrieval recall@1 on 6000 LIBERO frames (36× over random), available as a downstream initialisation.

The four contributions are not independently surprising — they are each modest extensions of established techniques to the under-studied flow-matching VLA paradigm. Their value is in the **comparative crossing**: under one shared protocol, with one open code repository and ckpt set, we map out which post-training techniques transfer cleanly across architectural paradigms and which need paradigm-specific adaptation.

We hope this work primes a more systematic cross-paradigm post-training literature for VLAs as more open backbones (Spirit, RDT, future π versions) become available.

All code, training logs, ckpts, and reproduction scripts are open at <https://github.com/lz-googlefycy/vla-lab>.